

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №1
по курсу «Логика и основы алгоритмизации
в инженерных задачах»
на тему «Простые структуры данных»

Выполнили ст. гр. 25BBB2:

Карпов А.А.

Долгов Л.А.

Приняли:

к.п.н., доцент Юрова О. В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Простые структуры данных

Цель работы: получение навыков алгоритмизации задач обработки данных и работы с простыми структурами данных — списками, вложенными списками и пользовательскими типами данных; освоение способов их описания, заполнения, обработки и поиска в них.

Лабораторное задание: выполнить пять заданий по работе с простыми структурами данных: обработка одномерного массива, заполнение массива случайными числами, создание массива произвольного размера, обработка двумерного массива и поиск в массиве структур.

Средства реализации: язык программирования Python 3.11, среда разработки PyCharm. Используются стандартные модули random (генерация псевдослучайных чисел) и dataclasses (описание пользовательского типа данных). Все пять заданий реализованы в одном модуле main.py и выполняются последовательно за один запуск программы.

Постановка задачи

Задание 1: написать программу, вычисляющую разницу между максимальным и минимальным элементами массива.

Задание 2: написать программу, реализующую инициализацию массива случайными числами.

Задание 3: написать программу, реализующую создание массива произвольного размера, вводимого с клавиатуры.

Задание 4: написать программу, вычисляющую сумму значений в каждом столбце (или строке) двумерного массива.

Задание 5: написать программу, осуществляющую поиск среди структур student структуры с заданными параметрами (фамилией, именем и т.д.).

Используемые структуры данных

Список (list) — упорядоченная изменяемая коллекция элементов, доступ к которым выполняется по индексу, начинающемуся с нуля. В отличие от массива в языках со статической типизацией, список в Python не требует предварительного объявления размера: память под него выделяется и

освобождается интерпретатором автоматически, а расширение выполняется методом `append()`. Именно поэтому список используется в работе и как статический массив фиксированной длины, и как динамический массив, размер которого определяется во время выполнения программы.

Двумерный массив представлен как вложенный список — список, элементами которого являются списки-строки. Обращение к элементу выполняется двумя индексами: первый задаёт номер строки, второй — номер столбца. Перебор такого списка внешним циклом `for` даёт строки матрицы целиком, что позволяет обрабатывать их встроенными функциями.

Структура (запись) реализована в виде класса, помеченного декоратором `@dataclass` из модуля `dataclasses`. Декоратор автоматически формирует конструктор класса по описанным полям и их значениям по умолчанию, поэтому такой класс является прямым аналогом структуры `struct` в языке Си. Класс `Student` объединяет разнотипные поля: фамилию, имя, название факультета (строки) и номер зачётной книжки (целое число). Доступ к полю выполняется через оператор «точка». Совокупность записей хранится в списке объектов — простейшей табличной структуре данных, к которой применим линейный поиск.

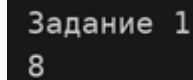
Ход работы

Задание 1. Разница между максимальным и минимальным элементами массива

Исходными данными служит список из десяти целых чисел, заданный непосредственно в тексте программы. С помощью генератора списка все его элементы приводятся к целому типу — это гарантирует корректность последующих сравнений, если данные будут получены из внешнего источника в виде строк.

Для нахождения экстремумов применяются встроенные функции `max()` и `min()`, каждая из которых выполняет один линейный проход по списку и возвращает наибольший и наименьший элементы соответственно. Искомая величина вычисляется как их разность. Трудоёмкость алгоритма линейная — $O(n)$.

Результаты работы программы показаны на рисунке 1.



```
Задание 1
8
```

Рисунок 1 — Результаты работы программы по заданию 1

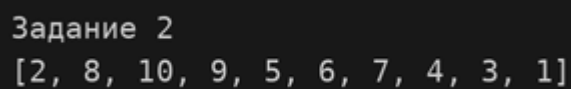
Проверка: наибольший элемент списка равен 9, наименьший равен 1, их разность составляет $9 - 1 = 8$. Результат совпал с выводом программы.

Задание 2. Заполнение массива случайными числами

Список заполняется псевдослучайными числами, которые формирует функция `randint()` модуля `random`: она возвращает целое число из указанного отрезка, включая обе его границы. В работе использован отрезок `[1, 10]`.

Заполнение выполняется циклом `while`, который повторяется до тех пор, пока длина списка не достигнет десяти элементов. Перед добавлением очередного числа проверяется условие «числа ещё нет в списке», поэтому повторяющиеся значения отбрасываются, а результатом работы является перестановка чисел от 1 до 10, расположенных в случайном порядке. Параллельно сформированные значения записываются в исходный список по индексу, который увеличивается на каждой успешной итерации.

Результаты работы программы показаны на рисунке 2.



```
Задание 2
[2, 8, 10, 9, 5, 6, 7, 4, 3, 1]
```

Рисунок 2 — Результаты работы программы по заданию 2

При каждом запуске программы порядок элементов различен, так как генератор псевдослучайных чисел инициализируется автоматически.

Задание 3. Создание массива произвольного размера

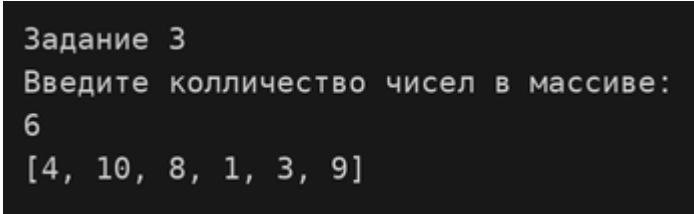
Количество элементов вводится с клавиатуры функцией `input()` и преобразуется к целому типу функцией `int()`. Размер списка, таким образом, становится известен только во время выполнения программы. Дополнительных действий по выделению памяти не требуется: список создаётся пустым и

расширяется методом `append()` по мере добавления элементов, а управление памятью полностью берёт на себя интерпретатор.

Заполнение выполняется тем же способом, что и в предыдущем задании: цикл `while` повторяется, пока длина списка не станет равна введённому значению, а условие отсутствия элемента в списке исключает повторы. Так как значения выбираются из отрезка $[1, 10]$ без повторов, корректный ввод ограничен количеством, не превышающим десяти элементов.

В примере на рисунке 3 введено количество, равное шести, и получен список из шести различных случайных чисел.

Результаты работы программы показаны на рисунке 3.



```
Задание 3
Введите количество чисел в массиве:
6
[4, 10, 8, 1, 3, 9]
```

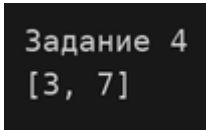
Рисунок 3 — Результаты работы программы по заданию 3

Задание 4. Суммы элементов строк двумерного массива

Двумерный массив задан вложенным списком размером 2×2 . Внешний цикл `for` последовательно перебирает строки матрицы, при этом переменная цикла на каждой итерации содержит очередной список-строку целиком. Сумма её элементов вычисляется встроенной функцией `sum()` и добавляется в результирующий список методом `append()`, после чего список сумм выводится на экран одной операцией.

Трудоёмкость алгоритма составляет $O(m \times n)$, где m — число строк, n — число столбцов матрицы. Условие задания допускает подсчёт сумм как по столбцам, так и по строкам; в работе реализован подсчёт по строкам.

Результаты работы программы показаны на рисунке 4.



```
Задание 4
[3, 7]
```

Рисунок 4 — Результаты работы программы по заданию 4

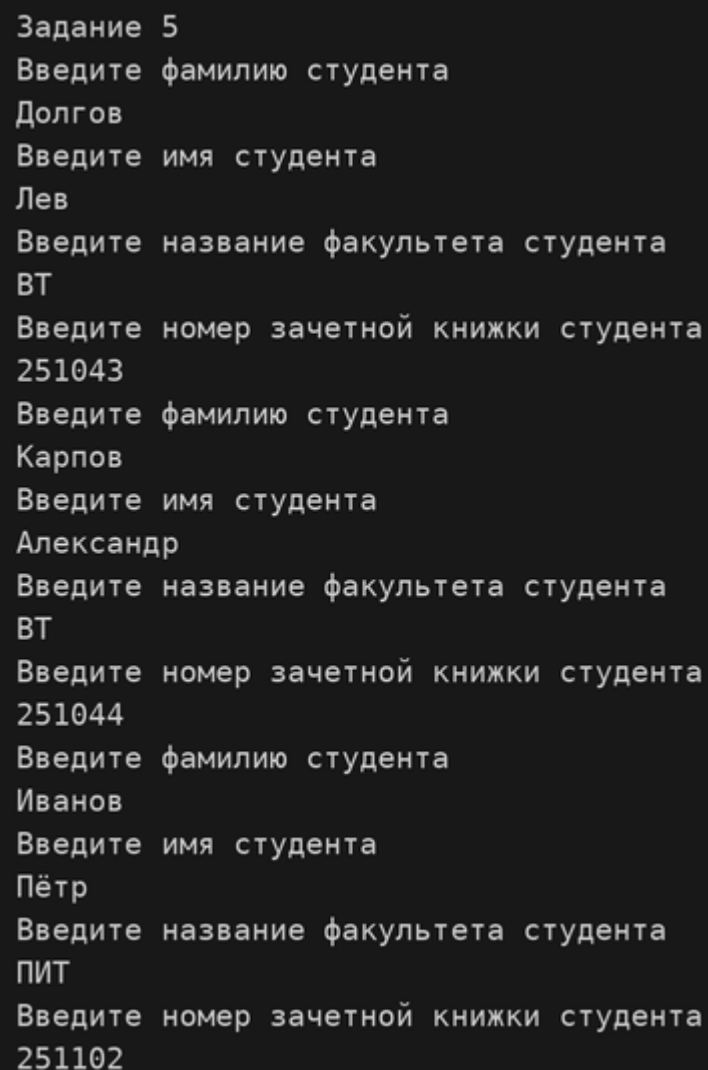
Проверка: сумма первой строки равна $1 + 2 = 3$, сумма второй строки равна $3 + 4 = 7$. Результаты совпали с выводом программы.

Задание 5. Поиск в массиве структур student

Для хранения сведений о студенте описан класс Student с декоратором @dataclass. Класс содержит четыре поля: фамилию, имя, название факультета и номер зачётной книжки, — каждому из которых задано значение по умолчанию. Декоратор автоматически формирует конструктор, поэтому объект создаётся без явного указания аргументов.

На основе класса создаётся список из трёх объектов. Заполнение выполняется циклом for: на каждой итерации с клавиатуры последовательно вводятся все четыре поля очередной записи, причём номер зачётной книжки приводится к целому типу. После завершения ввода список выводится на экран в виде таблицы записей.

Ввод исходных данных показан на рисунке 5.



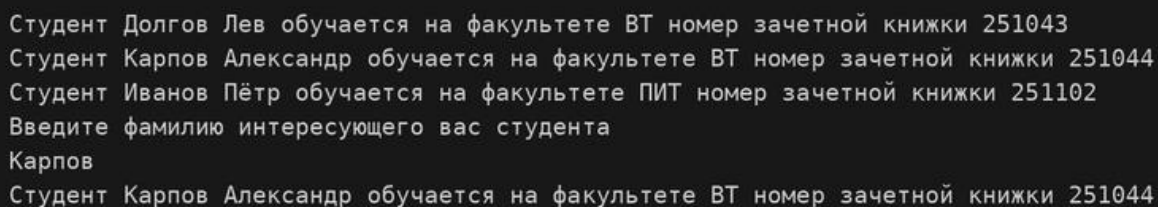
```
Задание 5
Введите фамилию студента
Долгов
Введите имя студента
Лев
Введите название факультета студента
ВТ
Введите номер зачетной книжки студента
251043
Введите фамилию студента
Карпов
Введите имя студента
Александр
Введите название факультета студента
ВТ
Введите номер зачетной книжки студента
251044
Введите фамилию студента
Иванов
Введите имя студента
Пётр
Введите название факультета студента
ПИТ
Введите номер зачетной книжки студента
251102
```

Рисунок 5 — Ввод сведений о студентах

Поиск выполняется по фамилии, введённой с клавиатуры, методом последовательного (линейного) перебора: список просматривается от начала до конца, и поле фамилии каждой записи сравнивается с введённым ключом. При совпадении на экран выводятся все поля найденной записи, устанавливается признак успешного поиска и цикл досрочно завершается оператором `break` — дальнейший просмотр не имеет смысла. Если после окончания перебора признак остался неустановленным, выводится сообщение об отсутствии записи в базе.

Трудоёмкость линейного поиска в худшем случае составляет $O(n)$. Он применим к неупорядоченному списку и не требует предварительной сортировки данных.

Вывод списка записей и результат поиска показаны на рисунке 6.



```
Студент Долгов Лев обучается на факультете ВТ номер зачетной книжки 251043
Студент Карпов Александр обучается на факультете ВТ номер зачетной книжки 251044
Студент Иванов Пётр обучается на факультете ПИТ номер зачетной книжки 251102
Введите фамилию интересующего вас студента
Карпов
Студент Карпов Александр обучается на факультете ВТ номер зачетной книжки 251044
```

Рисунок 6 — Вывод списка студентов и результат поиска

Введённой фамилии «Карпов» соответствует одна запись, все её поля выведены корректно. При вводе фамилии, отсутствующей в списке, программа выводит сообщение о том, что такого студента нет в базе.

Вывод

В ходе выполнения лабораторной работы были получены навыки алгоритмизации задач обработки данных и работы с простыми структурами данных средствами языка Python. Изучены создание и обработка списков и вложенных списков, заполнение списка псевдослучайными числами с исключением повторов, формирование списка произвольного размера, задаваемого во время выполнения программы, а также описание пользовательского типа данных с помощью декоратора `@dataclass` и организация линейного поиска по списку записей.

Все пять заданий выполнены, работа программы проверена на контрольных наборах данных: результаты, полученные при ручном расчёте, совпали с результатами работы программы.